

Lab & Activities Week 08 multi-agent-systems

Integrantes:

José Martínez-02220131016

Institución: Universidad de Santander

Pregrado: Ingeniería Se Sistemas

Docente: Jorge Galvis

03 de Octubre 2026

Activity 1

Activity 1 · Warm-up Concept Quiz10 MININDIVIDUALFORMATIVE

1. After a coordinator delegates to a `sub_agents` child, who replies to the user?

- ☐ The coordinator always replies, using the child's output
- ☐ The child agent — control has transferred to it
- ☐ Neither; the user must re-ask

Correct answer: B. `sub_agents` is a handoff: the child owns the conversation from that point.

2. You need the coordinator to combine a specialist's answer with another calculation before replying. Which mechanism fits?

- ☐ `sub_agents`
- ☐ `ParallelAgent` only
- ☐ `AgentTool`

Correct answer: C. `AgentTool` returns a string to the parent, which keeps control and can combine results.

3. What is the single most common cause of silent multi-agent mis-routing in `sub_agents`?

- ☒ A vague or missing description field on a child agent
- ☐ Using too many tools on the coordinator
- ☐ Using `SequentialAgent` instead of `ParallelAgent`

Correct answer: A. The router reads only the description field to decide where to send a request.

4. Which ADK workflow agent guarantees a fixed order with zero routing LLM calls?

- ☐ An orchestrator using `sub_agents`
- ☐ `SequentialAgent`
- ☐ `AgentTool`

Correct answer: B. `SequentialAgent` runs children in a fixed, deterministic order.

Check answersScore: 4 / 4

Activity 2

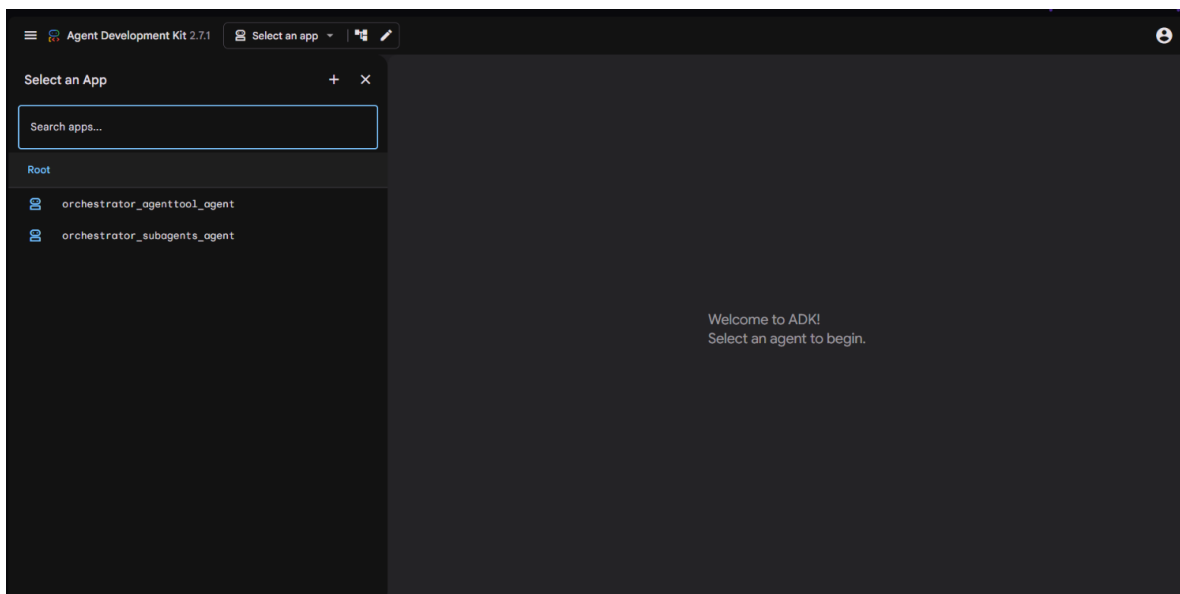
Ran all three orchestrator variants in week_8/ against the same prompt, "What does week 8 cover, and how many study hours should I budget if it's hard?":
orchestrator_subagents_agent/agent.py (`sub_agents` handoff),

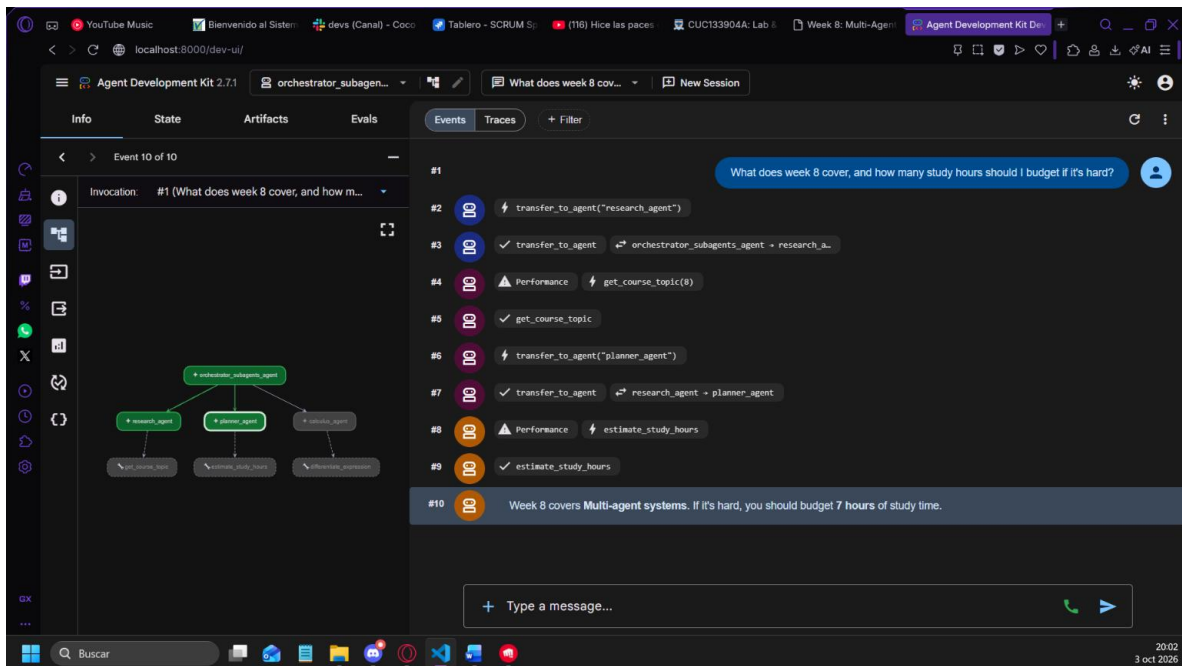
orchestrator_agenttool_agent/agent.py (AgentTool), and app.py's LangChain AgentTool-style equivalent.

Traced the sub_agents run end to end: the coordinator transferred to research_agent (get_course_topic), which then transferred control again, on its own, to planner_agent (estimate_study_hours) -- control changed hands twice, and the final reply came from planner_agent itself, not the original coordinator. ADK's transfer_to_agent is not limited to the coordinator; any agent in the tree can hand off to any other named agent, which is what let research_agent chain straight into planner_agent without going back through the coordinator first.

The AgentTool run never transferred control: orchestrator_agenttool_agent stayed the sole author of every event, calling research_agent and planner_agent as isolated tools (each returning a plain string) and assembling both results into its own final reply -- exactly the "parent stays in control and synthesizes" behavior the activity asks to watch for.

The LangChain app.py run reproduced the same AgentTool-style shape: two sequential tool calls (ask_research_agent, then ask_planner_agent), with the final AIMessage combining both into one answer -- still wrapped in the same block-list-with-signature structure noted back in Week 6/7, not a plain string.





Activity 3

Using the Corte 2 project problem you sketched in Week 6, design the agent team:

1. List 2–4 specialist roles and, for each, a one-sentence description good enough to route on.

R/

A coordinator (cubik_coordinator) classifies each request, routes it to a specialist, and keeps set_explanation_level.

Specialist Description

theory_agent Explains integral calculus concepts, definitions and theorems (e.g. why +C, the Fundamental Theorem), citing the knowledge base. Does not solve or check specific exercises.

solver_agent Solves a specific integral the student asks to be solved, step by step, naming the technique and the rule behind each step. Does not answer theory questions or check the student's own answers.

verifier_agent Checks the student's own answer to a specific integral against an exact SymPy computation, and explains the mistake if it is wrong. Does not solve exercises the student has not attempted.

progress_agent Summarizes the student's practice progress per technique, or resets their practice log.

2. For each specialist, decide: sub_agents handoff, AgentTool, or a fixed SequentialAgent/ParallelAgent step — and justify it.

R/

Specialist Delegation Why?

theory_agent sub_agents It answers the student directly; the coordinator has nothing to add to a concept explanation.

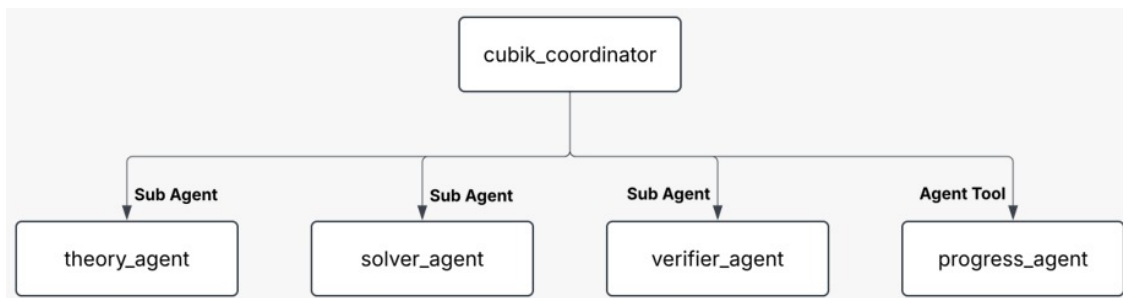
solver_agent sub_agents It answers the student directly with the step-by-step solution, computed with SymPy.

verifier_agent sub_agents It answers the student directly with the SymPy verdict and an explanation of the mistake.

progress_agent AgentTool (called by the coordinator) Returns short data the coordinator folds into its own reply.

3. Sketch the team as a small box diagram (coordinator at top, specialists below, labeled with the delegation type).

R/



By default, with sub_agents, the next message goes to the specialist that answered last, which must notice a topic switch and transfer. In our tests with a local model, specialists often failed to do this: they wrote the transfer as text or answered outside their scope. So, the specialists can't transfer and ADK sends every new message back to the coordinator. All routing lives in one agent that sees the full history, including follow-ups like "why did you use integration by parts?". The cost is one extra model call per turn.

Activity 4

Added a third specialist, calculus_agent, to orchestrator_subagents_agent/agent.py's sub_agents list (alongside research_agent and planner_agent) and to app.py's LangChain tool set (as ask_calculus_agent) -- both wrap a new differentiate_expression(expression, variable="x") tool built on sympy.diff, returning the symbolic derivative of a single-variable expression. Chosen as a direct, miniature stand-in for a specialist in the Corte 2 calculus-tutoring project sketched in Activity 3.

calculus_agent's description: "Computes the symbolic derivative of a single-variable calculus expression ... with respect to a given variable. Only for differentiation / derivative requests, never for schedule or study-time questions." The explicit exclusion clause was added after noticing research_agent/planner_agent's own descriptions only state what they DO, not what they must never be confused with.

5 test prompts, run for real against orchestrator_subagents_agent (python run_test.py subagents "..."):

Test 1 (schedule only): "What topic does week 10 cover?" -> transferred to research_agent -> get_course_topic(week=10) -> "Week 10 covers LangGraph II + advanced RAG." Routed correctly on the first hop.

Test 2 (study-time only): "I think RAG techniques will be a medium difficulty topic for me - how many hours should I budget?" -> transferred to planner_agent -> estimate_study_hours(topic="RAG techniques", difficulty="medium") -> "You should budget approximately 4 hours to study RAG techniques at a medium difficulty." Routed correctly on the first hop.

Test 3 (derivative only): "What is the derivative of $x^4 - 3x^2 + 5$ with respect to x ?" -> transferred to calculus_agent -> differentiate_expression(...) -> derivative $4x^3 - 6x$ -> "The derivative of $x^4 - 3x^2 + 5$ with respect to x is $4x^3 - 6x$." Routed correctly on the first hop, and the derivative itself is correct.

Test 4 (derivative, different phrasing, no explicit "derivative" + topic pairing): "Can you differentiate $\sin(x)\cos(x)$ for me?" -> transferred to calculus_agent -> differentiate_expression(" $\sin(x)\cos(x)$ ") -> raw result $-\sin(x)^2 + \cos(x)^2$, which the model simplified in its final reply to the equivalent identity $\cos(2x)$. Routed correctly on the first hop.

Test 5 (combined schedule + study-time, to confirm calculus_agent does not get pulled in by accident now that three specialists exist): "What does week 7 cover, and how many study hours should I budget for it at hard difficulty?" -> transferred to research_agent (get_course_topic: week 7 = "Google ADK") -> research_agent itself transferred to planner_agent (estimate_study_hours: hard = 7) -> final reply from planner_agent: "Week 7 covers Google ADK. At a hard difficulty, you should budget 7 study hours for it." 5 / 5 test prompts routed correctly on the first try, with no cross-talk from the new calculus_agent.

Confirmed the same specialist on the LangChain side too: asking app.py's orchestrator "What is the derivative of $x^4 - 3x^2 + 5$ with respect to x ?" correctly called ask_calculus_agent and returned the same $4x^3 - 6x$ result, combined into one final AIMessage.

Pending: commit the updated orchestrator_subagents_agent/agent.py, orchestrator_agenttool_agent/agent.py, and app.py under week-08/ to the course GitHub repository.

